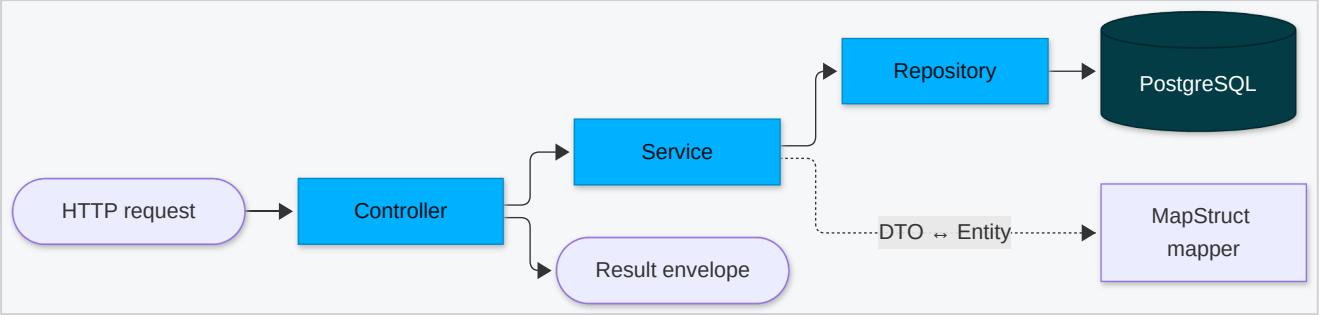
Back-end

`candex-api` is a layered Spring Boot application. This handbook covers how it is built; for the domain it serves, see Database & Model, and for the system around it, Architecture & Overview.

Request flow

Every request flows through the same layers. Controllers return a typed **`Result`** envelope; services hold the logic; repositories are Spring Data JPA; and the boundary between the JPA entities and the API shapes is crossed by **MapStruct** mappers.



Each layer has exactly one job:

Layer	Responsibility	Must not
Controller	Map HTTP to a method, enforce access (`@PreAuthorize`), return a `Result`. Thin.	Hold business logic or touch the database
Service (`service/core`)	The business logic: orchestration, transactions, rule checks, calling repositories, mapping entities ↔ DTOs, raising exceptions.	Know anything about HTTP
Repository	Spring Data JPA access to one aggregate — derived and custom queries.	Hold business rules
Entity	The JPA domain model (persisted state).	Leave the service boundary
DTO (`dto/request`, `dto/response`)	The API's input and output shapes.	Carry persistence concerns
Mapper (MapStruct)	Convert entities ↔ DTOs at the service boundary.	—
Cross-cutting (`security`, `exception`, `validation`, `config`, `job`, `result`)	Auth, structured errors, input validation, configuration, scheduled jobs, the response envelope.	—

The golden rule: entities never reach the controller, and controllers never hold logic.

The response envelope

Every endpoint returns a generic `Result<T>`:

```
{ "success": true, "data": { "...": "..." }, "error": null }
```

- `data` carries the payload on success; `error` carries a structured error on failure; `success` is the flag.
- Collection responses add an **`X-Total-Count`** header; creations add a **`Location`** header.

API surface

Resource paths are centralised in `ApiPaths` and exposed under the API base (for example `/api/drinks`). Around **45 endpoints** across ~14 resources (28 `GET`, 9 `POST`, 4 `DELETE`, 3 `PUT`, 1 `PATCH`):

Resource	Path
Drinks	`/drinks` (incl. bulk `import`, `import/diff`, `import/sync`)
Cans • Can formats	`/cans` • `/can-formats`
Brands	`/brands`
Flavors • Flavor categories	`/flavors` • `/flavor-categories`
Colors • Drink categories	`/colors` • `/drink-categories`
World Dex	`/world-dex`
Users • Unlocked drinks	`/users` • `/user-unlocked-drinks`
App versions • Reports	`/app-versions` • `/reports`
Admin SQL console	`/admin/...`

Interactive docs live at **Swagger UI** (`/swagger-ui.html`). The exact request/response contracts belong to the **API / Contracts** handbook *(planned)*.

Authorization

Endpoints are guarded with `@PreAuthorize` and `SecurityExpressions`:

Rule	Used for
`USER_OR_ADMIN`	Authenticated reads (the catalogue, the World Dex)
`ADMIN_ONLY`	Catalogue writes and admin operations
`PERMIT_ALL`	The few public endpoints
`@securityAccess.canAccessUserId(...)`	User-owned data — a user may only touch their own records

Identity itself comes from Firebase (see Architecture & Overview).

Services & business logic

The domain logic sits in `service/core` — one service per resource (`DrinkService`, `BrandService`, `WorldDexService`, `UserService`, …) — plus supporting services: `ImageService` (analysis), `UploaderService` (media), `ReportService`, and `SqlConsoleService`.

Error handling

Every exception extends a common `CustomException`, which **carries its own `HttpStatus`** and serialises into the `Result` envelope's `error` field (a `dto/response/Error` holding the message and status code). Exceptions fall into two families.

API errors (`exception/api`) — request and data problems:

Exception	HTTP	Raised when
`ElementNotFoundException`	404	A referenced entity does not exist
`InvalidFieldValueException`	400	A field has an invalid format or value
`MissingRequiredFieldException`	400	A required field is absent
`UnsupportedFieldException`	400	An unknown/unsupported field is used
`InvalidImportPayloadException`	400	A bulk-import payload is malformed
`SqlExecutionException`	400	A SQL-console query fails
`ImmutableFieldModificationException`	409	A create-only field is changed after creation
`UniqueConstraintViolationException`	409	A uniqueness constraint is violated
`MissingConfigurationException`	422	Required configuration (e.g. an app version) is missing

Business errors (`exception/business`) — domain-rule violations:

Exception	HTTP	Raised when
`MissingRequiredCanException`	400	A drink is saved without any can
`MissingRequiredLocalizationException`	400	A drink is saved without any localization
`InvalidDexPositionException`	400	A World Dex position is out of range
`InvalidEditionConfigurationException`	400	An edition / variant is misconfigured
`DrinkAlreadyHasPredecessorException`	409	A drink is linked as predecessor to two drinks
`ImageAnalysisException`	422	Image analysis fails

In short: `api` exceptions describe what is wrong with the request; `business` exceptions describe which domain rule was broken.

Validation

Request DTOs are validated with Bean Validation, wired through `ValidationConfig`. Invalid input surfaces as an `api` error rather than reaching the service layer.

Scheduled jobs

Two background jobs run on a schedule:

- `UnverifiedUserCleanupJob` — removes users who never verified their account.
- `WorldDexHistoryPruneJob` — trims old World Dex history.

Configuration

Configuration lives in `config` and is profile-aware (`dev` / `prod`): `OpenApiConfig` (Swagger), `CorsConfig` (allowed origins), `JacksonConfig` (JSON), `SqlConsoleConfig`, and `ValidationConfig`. To run it locally, see Local Installation; environment specifics belong to the **Deployment** handbook *(planned)*.

Conventions

- Controllers return `ResponseEntity<Result<T>>`; paths come from `ApiPaths` constants.
- Entities never leave the service boundary — DTOs (`dto/request`, `dto/response`) cross it via MapStruct mappers.
- Every endpoint declares its access with `@PreAuthorize`.
- Errors are structured exceptions (`api` / `business`) surfaced through the `Result` envelope.